

LC 1579: Remove Max number of edges to keep graph fully Traversable

Authored by: Arpit Raj, LNMIIT Jaipur | Date: August 4, 2026

Uber

Adobe

Problem Understanding & Legend

T1 (wavy line) → travel by A

T3 (solid line) → travel by both A and B

T2 (dashed line) → travel by B

Goal: need to remove edges such that still fully traversable by A and B.

Example Note: here we can remove both wavy type bw $1 \rightarrow 2$ and $1 \rightarrow 3$, so **O/p = 2**

Observations

Obv 1: T3 edges are most usable.

Obv 2: alice and bob movement are independent hence...

Algorithm Steps

- S1:** need to separate disjoint sets for both alice and bob
- S2:** traverse type 3 edges first, if already cycle remove kr do otherwise alice aur bob me union
- S3:** traverse type 1 edges, do only Alice DSU. If cycle remove, else union in only alice
- S4:** same as S3 but for bob
- S5:** verify if no. of comp = 1 for both alice and bob, else -1

C++ Implementation

```

class dsu {
public:
    vector<int> parent;
    vector<int> size;

    dsu(int n) {
        parent.resize(n + 1);
        size.resize(n + 1);
        for(int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int findpar(int node) {
        if(parent[node] == node) {
            return node;
        }
        parent[node] = findpar(parent[node]);
        return parent[node];
    }

    void merge(int u, int v) {
        int pu = findpar(u);
        int pv = findpar(v);

        if(pu == pv) { return; }

        if(size[pu] < size[pv]) {
            swap(pu, pv);
        }

        size[pu] += size[pv];
        parent[pv] = parent[pu];
    }
};

class Solution {
public:
    int maxNumEdgesToRemove(int n, vector<vector<int>>& edges) {
        dsu dsa(n);
        dsu dsb(n);
        int ans = 0;

        for(auto &it : edges) {
            int u = it[1];
            int v = it[2];
            int type = it[0];

            if(type == 3) {
                if(dsa.findpar(u) == dsa.findpar(v)) {
                    ans++;
                }
            }
        }
    }
};

```

```

        } else {
            dsa.merge(u, v);
            dsb.merge(u, v);
        }
    }
}

for(auto &it : edges) {
    int u = it[1];
    int v = it[2];
    int type = it[0];

    if(type == 1) {
        if(dsa.findpar(u) == dsa.findpar(v)) {
            ans++;
        } else {
            dsa.merge(u, v);
        }
    }
}

for(auto &it : edges) {
    int u = it[1];
    int v = it[2];
    int type = it[0];

    if(type == 2) {
        if(dsb.findpar(u) == dsb.findpar(v)) {
            ans++;
        } else {
            dsb.merge(u, v);
        }
    }
}

unordered_set<int> sta;
unordered_set<int> stb;

for(int i = 1; i <= n; i++) {
    sta.insert(dsa.findpar(i));
}

for(int i = 1; i <= n; i++) {
    stb.insert(dsb.findpar(i));
}

if(sta.size() != 1 || stb.size() != 1) return -1;
else return ans;
}
};

```